

Homework 4: Generative Automatic Prompt Optimization & Interpretability

This homework guides you through building an "Automatic Prompt Engineer" (APE) — a system where an LLM iteratively improves its own prompts based on error analysis. You will then use white-box interpretability techniques to understand *why* the optimized prompt works better.

Total Points: 20

Learning Objectives

By completing this homework, you will:

- Understand how to build an automatic prompt optimization system using LLMs
- Learn to evaluate prompt performance systematically
- Gain experience with white-box interpretability techniques (attention patterns, confidence analysis)
- Analyze why certain prompts work better than others
- Practice error analysis and iterative improvement

Expected Outcomes

After completing this homework, you should be able to:

1. Generate and evaluate prompts programmatically
2. Use meta-prompting to improve prompts iteratively
3. Extract and visualize attention patterns from transformer models
4. Analyze model confidence through logit analysis
5. Draw meaningful insights from interpretability tools

Instructions:

1. Complete all tasks in this notebook
2. Ensure your code runs without errors
3. Submit both this notebook and any additional files created
4. Write clear explanations for your approach
5. Fill in the `## YOUR_CODE_STARTS_HERE` / `## YOUR_CODE_ENDS_HERE` sections
6. Run the answer checkpoints after each task to verify your implementation

Setup

```
1 # Install required packages
2 %pip install -q transformers torch datasets
3 %pip install -q matplotlib seaborn numpy
4
```

```
1 import torch
2 import torch.nn.functional as F
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from transformers import AutoTokenizer, AutoModelForCausalLM
7 from typing import List, Dict, Tuple, Optional
8 import random
9 from collections import Counter
10 import re
11
12 # Set random seeds for reproducibility
13 torch.manual_seed(42)
14 np.random.seed(42)
15 random.seed(42)
16
17 # Device setup
18 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
19 print(f"Using device: {device}")
20
```

Using device: cuda

```
1 from huggingface_hub import login
2 login()
```

```

1 # Using Gemma-2B-IT - better at arithmetic tasks than Phi-2
2 # If you prefer Phi-2, uncomment the alternative code below
3 print("Loading google/gemma-2b-it (excellent for math, supports output_attentions)...")
4 model_name = "google/gemma-2b-it"
5
6 tokenizer = AutoTokenizer.from_pretrained(model_name)
7 # Gemma needs pad_token set
8 if tokenizer.pad_token is None:
9     tokenizer.pad_token = tokenizer.eos_token
10
11 model = AutoModelForCausalLM.from_pretrained(
12     model_name,
13     dtype=torch.float16 if device.type == "cuda" else torch.float32,
14     device_map="auto" if device.type == "cuda" else None,
15     # Force eager attention for output_attentions support
16     attn_implementation="eager"
17 )
18 if device.type == "cpu":
19     model = model.to(device)
20 model.eval()
21 print("Model loaded successfully!")

```

Loading google/gemma-2b-it (excellent for math, supports output_attentions)...

config.json: 100% 627/627 [00:00<00:00, 58.9kB/s]

tokenizer_config.json: 100% 34.2k/34.2k [00:00<00:00, 4.10MB/s]

tokenizer.json: 100% 17.5M/17.5M [00:00<00:00, 21.6MB/s]

special_tokens_map.json: 100% 636/636 [00:00<00:00, 75.2kB/s]

model.safetensors.index.json: 100% 13.5k/13.5k [00:00<00:00, 1.28MB/s]

Download complete: 100% 5.01G/5.01G [01:03<00:00, 262MB/s]

Fetching 2 files: 100% 2/2 [00:53<00:00, 53.15s/it]

Loading weights: 100% 164/164 [00:15<00:00, 8.40it/s, Materializing param=model.norm.weight]

generation_config.json: 100% 137/137 [00:00<00:00, 2.31kB/s]

Model loaded successfully!

Dataset: Arithmetic Word Problems

We'll use a synthetic dataset of simple arithmetic word problems. This task is perfect for prompt optimization because:

1. Clear right/wrong answers (easy to evaluate)
2. The way you phrase the instruction matters a lot
3. Errors are interpretable (wrong numbers, wrong operations)

```

1 # Synthetic dataset of arithmetic word problems
2 def generate_arithmetic_dataset(n_samples: int = 100, seed: int = 42) -> List[Dict]:
3     """Generate synthetic arithmetic word problems."""
4     random.seed(seed)
5     np.random.seed(seed)
6
7     templates = [
8         ("Alice has {a} apples. Bob gives her {b} more apples. How many apples does Alice have now?",
9          lambda a, b: a + b),
10        ("There are {a} birds on a tree. {b} birds fly away. How many birds are left?",
11         lambda a, b: a - b),
12        ("A store has {a} boxes. Each box contains {b} items. How many items are there in total?",
13         lambda a, b: a * b),
14        ("Tom has {a} candies. He wants to share them equally among {b} friends. How many candies does each friend get?",
15         lambda a, b: a // b),
16        ("A class has {a} students. {b} more students join. What is the total number of students?",
17         lambda a, b: a + b),
18        ("A farmer has {a} cows. He sells {b} of them. How many cows does he have left?",
19         lambda a, b: a - b),
20    ]
21
22    dataset = []
23    for i in range(n_samples):
24        template, operation = random.choice(templates)
25
26        # Generate appropriate numbers based on operation
27        if 'fly away' in template or 'sells' in template or 'left' in template:
28            # Subtraction: ensure a >= b
29            a = random.randint(10, 50)
30            b = random.randint(1, a)
31        elif 'share' in template or 'equally' in template:

```

```

32         # Division: ensure clean division
33         b = random.randint(2, 10)
34         a = b * random.randint(2, 10)
35     elif 'boxes' in template or 'contains' in template:
36         # Multiplication: keep numbers small
37         a = random.randint(2, 12)
38         b = random.randint(2, 12)
39     else:
40         # Addition
41         a = random.randint(5, 50)
42         b = random.randint(5, 50)
43
44     question = template.format(a=a, b=b)
45     answer = str(operation(a, b))
46
47     dataset.append({
48         "question": question,
49         "answer": answer
50     })
51
52     return dataset
53
54 # Generate dataset
55 full_dataset = generate_arithmetic_dataset(n_samples=120, seed=42)
56
57 # Split into train (for error analysis) and validation (for evaluation)
58 train_data = full_dataset[:60]
59 val_data = full_dataset[60:100]
60 test_data = full_dataset[100:]
61
62 print(f"Train set: {len(train_data)} examples")
63 print(f"Validation set: {len(val_data)} examples")
64 print(f"Test set: {len(test_data)} examples")
65 print("\nSample questions:")
66 for i in range(3):
67     print(f"  Q: {train_data[i]['question']}")
68     print(f"  A: {train_data[i]['answer']}\n")
69

```

Train set: 60 examples

Validation set: 40 examples

Test set: 20 examples

Sample questions:

Q: A farmer has 17 cows. He sells 1 of them. How many cows does he have left?

A: 16

Q: A farmer has 27 cows. He sells 8 of them. How many cows does he have left?

A: 19

Q: There are 18 birds on a tree. 4 birds fly away. How many birds are left?

A: 14

✓ TASK 1: The Evaluation Framework (5 points)

Build the infrastructure to evaluate how well a prompt performs on our arithmetic task.

1.1: Generate Response — Implement a function to generate model responses given a prompt template

1.2: Evaluate Prompt — Implement evaluation that returns accuracy and failed cases

1.3: Baseline Testing — Establish baseline performance with a naive prompt

✓ Task 1.1: Generate Response Function

Goal: Implement a function that generates model responses given a prompt template and question.

Requirements:

- Format the prompt template by inserting the question into the `{question}` placeholder
- Tokenize the formatted prompt and prepare it for the model (move to correct device)
- Generate a response using the model with greedy decoding (deterministic)
- Extract only the newly generated tokens (not the input prompt tokens)
- Decode the generated tokens back to text
- Return a dictionary with at least a `'response'` key containing the generated text
- Optionally include `'logits'` and `'attentions'` in the dictionary if requested via function parameters

Important considerations:

- For decoder-only models, `outputs.sequences` contains both input and generated tokens - you need to extract only the new tokens
- Handle `pad_token_id` appropriately (use `eos_token_id` if `pad_token_id` is None)
- Clean up the response text (strip whitespace, handle newlines appropriately)

Expected return format: A dictionary with at least `{'response': str}`. The response should be a non-empty string containing the model's generated text.

Success criteria:

- Function returns a dictionary
- Dictionary contains a `'response'` key with a non-empty string
- For arithmetic problems, the response should ideally contain a number

```

1 def generate_response(
2     model,
3     tokenizer,
4     prompt_template: str,
5     question: str,
6     max_new_tokens: int = 200, # Reduced for arithmetic (just need a number)
7     return_logits: bool = False,
8     return_attentions: bool = False
9 ) -> Dict:
10     """
11     Generate a response from the model given a prompt template and question.
12
13     Args:
14         model: The language model
15         tokenizer: The tokenizer
16         prompt_template: A string with {question} placeholder (e.g., "Solve: {question}")
17         question: The actual question to insert
18         max_new_tokens: Maximum tokens to generate
19         return_logits: Whether to return the logits
20         return_attentions: Whether to return attention weights
21
22     Returns:
23         Dictionary with 'response' (str), and optionally 'logits' and 'attentions'
24     """
25     ## YOUR_CODE_STARTS_HERE
26     prompt = prompt_template.format(question = question)
27     tok_prompt = tokenizer(prompt, return_tensors = "pt", padding = True).to(device)
28
29     if tokenizer.pad_token_id is not None:
30         pad_token_id = tokenizer.pad_token_id
31     else:
32         pad_token_id = tokenizer.eos_token_id
33
34     with torch.no_grad():
35         outputs = model.generate(
36             **tok_prompt,
37             max_new_tokens = max_new_tokens,
38             do_sample = False,
39             pad_token_id = pad_token_id,
40             return_dict_in_generate = True,
41             output_scores = return_logits,
42             output_attentions = return_attentions
43         )
44
45     prompt_length = tok_prompt["input_ids"].shape[1]
46     gen_ids = outputs.sequences[:, prompt_length:]
47
48     generated = tokenizer.decode(gen_ids[0], skip_special_tokens = True).strip()
49     result = {"response": generated}
50
51     if return_logits:
52         result["logits"] = outputs.scores
53
54     if return_attentions:
55         result["attentions"] = outputs.attentions
56
57     return result
58     ## YOUR_CODE_ENDS_HERE
59

```

```

1 # Test the generate_response function
2 test_prompt = "Answer the math question: {question}"
3 test_question = "Alice has 5 apples. Bob gives her 3 more apples. How many apples does Alice have now?"
4
5 result = generate_response(model, tokenizer, test_prompt, test_question)

```

```

6 print(f"Prompt template: {test_prompt}")
7 print(f"Question: {test_question}")
8 print(f"Response: {result['response']}")
9 ## RESULT_CHECKING_POINT -> Should output a number (ideally '8')
10

```

Prompt template: Answer the math question: {question}

Question: Alice has 5 apples. Bob gives her 3 more apples. How many apples does Alice have now?

Response: Alice starts with 5 apples, and Bob gives her 3 more apples, so she ends up with $5 + 3 = 8$ apples.

Therefore, Alice has 8 apples now.

```

1 # Answer Checkpoint: Task 1.1 - Generate Response
2 print("="*70)
3 print("VALIDATING: generate_response function")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 # Test 1: Function returns dictionary
10 try:
11     test_result = generate_response(model, tokenizer, "Test: {question}", "What is 2+2?")
12     if isinstance(test_result, dict):
13         print("✓ PASS: Function returns dictionary")
14         checks_passed += 1
15     else:
16         print(f"X FAIL: Function should return a dictionary, got {type(test_result)}")
17         checks_failed += 1
18 except Exception as e:
19     print(f"X FAIL: Function raised an error: {e}")
20     checks_failed += 1
21     test_result = None
22
23 # Test 2: Dictionary has 'response' key
24 if test_result is not None:
25     try:
26         if 'response' in test_result:
27             print("✓ PASS: Dictionary contains 'response' key")
28             checks_passed += 1
29         else:
30             print(f"X FAIL: Dictionary should contain 'response' key. Keys found: {list(test_result.keys())}")
31             checks_failed += 1
32     except Exception as e:
33         print(f"X FAIL: Error checking 'response' key: {e}")
34         checks_failed += 1
35
36 # Test 3: Response is non-empty string
37 if test_result is not None and 'response' in test_result:
38     try:
39         response = test_result['response']
40         if isinstance(response, str) and len(response) > 0:
41             print("✓ PASS: Response is a non-empty string")
42             checks_passed += 1
43         else:
44             print(f"X FAIL: Response should be a non-empty string. Got: {type(response)}, length: {len(response)} if is")
45             checks_failed += 1
46     except Exception as e:
47         print(f"X FAIL: Error checking response type: {e}")
48         checks_failed += 1
49
50 # Test 4: Response contains a number (for arithmetic problems)
51 if test_result is not None and 'response' in test_result:
52     try:
53         response = test_result['response']
54         import re
55         has_number = bool(re.search(r'\d+', response))
56         if has_number:
57             print("✓ PASS: Response contains a number")
58             checks_passed += 1
59         else:
60             print("⚠ WARNING: Response doesn't contain a number - this might be okay for some prompts")
61             print(f"    Response was: '{response[:100]}...'"
62     except Exception as e:
63         print(f"⚠ WARNING: Could not check for numbers: {e}")
64
65 # Test 5: Test with the actual test case from the notebook
66 try:
67     test_prompt = "Answer the math question: {question}"
68     test_question = "Alice has 5 apples. Bob gives her 3 more apples. How many apples does Alice have now?"
69     result = generate_response(model, tokenizer, test_prompt, test_question)
70     if isinstance(result, dict) and 'response' in result:

```

```

71     print("✓ PASS: Function works with test case from notebook")
72     checks_passed += 1
73     # Check if response contains the expected answer (8)
74     response = result['response']
75     if '8' in response or 'eight' in response.lower():
76         print("✓ PASS: Response contains expected answer (8)")
77         checks_passed += 1
78     else:
79         print(f"⚠ INFO: Response doesn't clearly contain '8', but this may vary by model")
80         print(f"    Response: '{response[:100]}...'")
81     else:
82         print("X FAIL: Function doesn't work with test case")
83         checks_failed += 1
84 except Exception as e:
85     print(f"X FAIL: Error with test case: {e}")
86     checks_failed += 1
87
88 print("\n" + "="*70)
89 if checks_failed == 0:
90     print(f"All checks passed! ✓ ({checks_passed} checks)")
91 else:
92     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
93     print("Please review your implementation and try again.")
94 print("="*70)
95

```

```

=====
VALIDATING: generate_response function
=====
✓ PASS: Function returns dictionary
✓ PASS: Dictionary contains 'response' key
✓ PASS: Response is a non-empty string
✓ PASS: Response contains a number
✓ PASS: Function works with test case from notebook
✓ PASS: Response contains expected answer (8)

=====
All checks passed! ✓ (6 checks)
=====

```

Task 1.2: Evaluate Prompt Function

Goal: Implement functions to extract numbers from text and evaluate prompt performance on a dataset.

Part A: `extract_number` function

Extract the first numeric answer from a text response. The model might respond in various formats (e.g., `"8"`, `"The answer is 8"`, `"8 apples"`, `"Alice has 8 apples"`). The function should handle these cases and return the first number found as a string, or `None` if no number is present.

Expected return: A string containing the number (e.g., `"8"`) or `None` if no number found.

Part B: `evaluate_prompt` function

Evaluate how well a prompt performs on a dataset by:

- Running the prompt on each example using `generate_response()`
- Extracting the predicted number from each response using `extract_number()`
- Comparing predictions to target answers (exact string match)
- Calculating accuracy as `correct_predictions / total_examples`
- Collecting failed cases with information about each error

Expected return format: A tuple `((float, List[Dict]))` where:

- First element is accuracy (0.0 to 1.0)
- Second element is a list of dictionaries, each containing failed case information with keys: `'question'`, `'prediction'`, `'target'`, `'raw_response'`

Success criteria:

- `extract_number` handles various text formats correctly
- `evaluate_prompt` returns a tuple of (float, list)
- Accuracy is between 0.0 and 1.0
- Failed cases list contains dictionaries with the required keys

```

1 def extract_number(text: str) -> Optional[str]:
2     """
3     Extract the first number from a text response.
4     Handles cases like '8', 'The answer is 8', '8 apples', etc.

```

```

5     """
6     # Clean up text - remove special characters that might confuse extraction
7     text = text.strip()
8
9     # Remove common prefixes
10    prefixes = ["Answer:", "The answer is", "Answer is", "The answer:", "Result:", "Result is"]
11    for prefix in prefixes:
12        if text.lower().startswith(prefix.lower()):
13            text = text[len(prefix):].strip()
14
15    # Find all numbers in the text
16    numbers = re.findall(r'?\d+', text)
17    if numbers:
18        # Return the first number that looks like an answer (not too large, positive for our problems)
19        for num in numbers:
20            num_int = int(num)
21            # For arithmetic word problems, answers are typically reasonable positive numbers
22            if 0 <= num_int <= 10000: # Reasonable range for our problems
23                return num
24        # If no reasonable number found, return first number anyway
25    return numbers[0]
26    return None
27
28
29 def evaluate_prompt(
30     model,
31     tokenizer,
32     prompt_template: str,
33     dataset: List[Dict],
34     verbose: bool = False
35 ) -> Tuple[float, List[Dict]]:
36     """
37     Evaluate a prompt template on a dataset.
38
39     Args:
40         model: The language model
41         tokenizer: The tokenizer
42         prompt_template: Template with {question} placeholder
43         dataset: List of {'question': str, 'answer': str} dicts
44         verbose: Whether to print progress
45
46     Returns:
47         Tuple of (accuracy: float, failed_cases: List[Dict])
48         failed_cases contains {'question', 'prediction', 'target'} for each error
49     """
50     ## YOUR_CODE_STARTS_HERE
51     failed_cases = []
52     correct = 0
53
54     for idx, exp in enumerate(dataset):
55         question = exp["question"]
56         target = exp["answer"]
57
58         generated = generate_response( # get response from question and prompt
59             model = model,
60             tokenizer = tokenizer,
61             prompt_template = prompt_template,
62             question = question,
63             max_new_tokens = 10
64         )
65         if generated is None:
66             failed_cases.append({
67                 "question": question,
68                 "prediction": None,
69                 "target": target,
70                 "raw_response": None
71             })
72             continue
73
74         raw_response = generated.get("response", "")
75
76         predicted = extract_number(raw_response)
77         if predicted == target: # compare target with raw response
78             correct += 1
79         else:
80             failed_cases.append({
81                 "question": question,
82                 "prediction": predicted,
83                 "target": target,
84                 "raw_response": raw_response
85             })
86

```

```

87 accuracy = correct / len(dataset)
88
89 ## YOUR_CODE_ENDS_HERE
90
91 return accuracy, failed_cases
92

```

```

1 # Test the evaluate_prompt function on a small subset
2 test_prompt = "{question}"
3 accuracy, failures = evaluate_prompt(model, tokenizer, test_prompt, val_data[:10], verbose=True)
4 print(f"\nAccuracy: {accuracy:.2%}")
5 print(f"Number of failures: {len(failures)}")
6 if failures:
7     print(f"\nSample failure:")
8     print(f" Question: {failures[0]['question']}")
9     print(f" Predicted: {failures[0]['prediction']}")
10    print(f" Target: {failures[0]['target']}")
11

```

Accuracy: 0.00%
Number of failures: 10

Sample failure:
Question: Tom has 6 candies. He wants to share them equally among 2 friends. How many candies does each friend get?
Predicted: None
Target: 3

```

1 # Answer Checkpoint: Task 1.2 - Evaluate Prompt
2 print("="*70)
3 print("VALIDATING: extract_number and evaluate_prompt functions")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 # Test 1: extract_number handles various formats
10 test_cases = [
11     ("8", "8"),
12     ("The answer is 8", "8"),
13     ("8 apples", "8"),
14     ("Alice has 8 apples", "8"),
15     ("Result: 42", "42"),
16     ("No number here", None),
17     ("", None),
18 ]
19
20 print("\nTesting extract_number function:")
21 for text, expected in test_cases:
22     try:
23         result = extract_number(text)
24         if result == expected:
25             print(f"✓ PASS: extract_number('{text[:30]}...') = {result}")
26             checks_passed += 1
27         else:
28             print(f"X FAIL: extract_number('{text[:30]}...') = {result}, expected {expected}")
29             checks_failed += 1
30     except Exception as e:
31         print(f"X FAIL: extract_number('{text[:30]}...') raised error: {e}")
32         checks_failed += 1
33
34 # Test 2: evaluate_prompt returns tuple
35 print("\nTesting evaluate_prompt function:")
36 try:
37     test_prompt = "{question}\nAnswer:"
38     accuracy, failures = evaluate_prompt(model, tokenizer, test_prompt, val_data[:5], verbose=False)
39
40     # Check return type
41     if isinstance(accuracy, (int, float)) and isinstance(failures, list):
42         print("✓ PASS: evaluate_prompt returns tuple of (number, list)")
43         checks_passed += 1
44     else:
45         print(f"X FAIL: evaluate_prompt should return (float, list), got ({type(accuracy)}, {type(failures)})")
46         checks_failed += 1
47
48     # Check accuracy range
49     if 0.0 <= accuracy <= 1.0:
50         print(f"✓ PASS: Accuracy is in valid range [0.0, 1.0]: {accuracy:.2%}")
51         checks_passed += 1
52     else:
53         print(f"X FAIL: Accuracy should be between 0.0 and 1.0, got {accuracy}")
54         checks_failed += 1

```



```

55
56 # Check failed cases structure
57 if len(failures) > 0:
58     first_failure = failures[0]
59     required_keys = ['question', 'prediction', 'target', 'raw_response']
60     if all(key in first_failure for key in required_keys):
61         print("✓ PASS: Failed cases have correct structure")
62         checks_passed += 1
63     else:
64         missing = [k for k in required_keys if k not in first_failure]
65         print(f"✗ FAIL: Failed cases missing keys: {missing}")
66         checks_failed += 1
67 else:
68     print("ℹ INFO: No failures found (this is okay if accuracy is 100%)")
69     checks_passed += 1
70
71 except Exception as e:
72     print(f"✗ FAIL: evaluate_prompt raised error: {e}")
73     checks_failed += 1
74
75 print("\n" + "="*70)
76 if checks_failed == 0:
77     print(f"All checks passed! ✓ ({checks_passed} checks)")
78 else:
79     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
80     print("Please review your implementation and try again.")
81 print("="*70)
82

```

```

=====
VALIDATING: extract_number and evaluate_prompt functions
=====

Testing extract_number function:
✓ PASS: extract_number('8...') = 8
✓ PASS: extract_number('The answer is 8...') = 8
✓ PASS: extract_number('8 apples...') = 8
✓ PASS: extract_number('Alice has 8 apples...') = 8
✓ PASS: extract_number('Result: 42...') = 42
✓ PASS: extract_number('No number here...') = None
✓ PASS: extract_number('...') = None

Testing evaluate_prompt function:
✓ PASS: evaluate_prompt returns tuple of (number, list)
✓ PASS: Accuracy is in valid range [0.0, 1.0]: 60.00%
✓ PASS: Failed cases have correct structure

=====
All checks passed! ✓ (10 checks)
=====

```

Task 1.3: Baseline Evaluation

Goal: Test multiple baseline prompts to establish a starting point for optimization.

Requirements:

- Define a list of simple, naive prompt templates to test (at least 3-5 prompts)
- Evaluate each prompt on the validation set using `evaluate_prompt()`
- Store results (accuracy and failures) for each prompt in a dictionary
- Select the prompt with the highest accuracy as the starting point for optimization

What to look for:

- Different prompts will have different accuracies
- The best baseline gives us a reference point - we want to improve upon it
- Typical baseline accuracies for arithmetic problems: 20-60% (varies by model and prompt quality)

Success criteria:

- At least 3-5 baseline prompts tested
- Best baseline accuracy > 0% (model is working)
- Results dictionary structure is correct with 'accuracy' and 'failures' keys
- Best baseline prompt selected and stored

```

1 # Define several baseline prompts to test (intentionally weak to show optimization improvement)
2 baseline_prompts = [
3     "{question}", # Just the question, no instruction
4     "{question}?", # Just question mark
5     "Here: {question}", # Ambiguous
6     "{question} answer", # Unclear format

```

```

7     "Math: {question}", # Minimal context
8 ]
9
10 print("Evaluating baseline prompts on validation set...\n")
11 print("=" * 70)
12
13 baseline_results = {}
14 for prompt in baseline_prompts:
15     accuracy, failures = evaluate_prompt(model, tokenizer, prompt, val_data)
16     baseline_results[prompt] = {'accuracy': accuracy, 'failures': failures}
17     print(f"Prompt: '{prompt[:50]}...' " if len(prompt) > 50 else f"Prompt: '{prompt}'")
18     print(f"Accuracy: {accuracy:.2%} ({int(accuracy * len(val_data))}/{len(val_data)})")
19     print("-" * 70)
20
21 # Select the best baseline as our starting point
22 best_baseline_prompt = max(baseline_results.keys(), key=lambda p: baseline_results[p]['accuracy'])
23 best_baseline_accuracy = baseline_results[best_baseline_prompt]['accuracy']
24 best_baseline_failures = baseline_results[best_baseline_prompt]['failures']
25
26 print(f"\nBest baseline prompt: '{best_baseline_prompt}'")
27 print(f"Best baseline accuracy: {best_baseline_accuracy:.2%}")
28

```

Evaluating baseline prompts on validation set...

```

=====
Prompt: '{question}'
Accuracy: 5.00% (2/40)
-----
Prompt: '{question}?'
Accuracy: 7.50% (3/40)
-----
Prompt: 'Here: {question}'
Accuracy: 5.00% (2/40)
-----
Prompt: '{question} answer'
Accuracy: 70.00% (28/40)
-----
Prompt: 'Math: {question}'
Accuracy: 0.00% (0/40)
-----

```

Best baseline prompt: '{question} answer'
Best baseline accuracy: 70.00%

```

1 # Answer Checkpoint: Task 1.3 - Baseline Evaluation
2 print("="*70)
3 print("VALIDATING: Baseline evaluation results")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 # Check if baseline_results exists and has correct structure
10 try:
11     if 'baseline_results' in globals():
12         print("✓ PASS: baseline_results variable exists")
13         checks_passed += 1
14
15     # Check that at least 3 prompts were tested
16     if len(baseline_results) >= 3:
17         print(f"✓ PASS: At least 3 baseline prompts tested ({len(baseline_results)} prompts)")
18         checks_passed += 1
19     else:
20         print(f"X FAIL: Should test at least 3 prompts, found {len(baseline_results)}")
21         checks_failed += 1
22
23     # Check structure of results
24     for prompt, result in baseline_results.items():
25         if isinstance(result, dict) and 'accuracy' in result and 'failures' in result:
26             checks_passed += 1
27             break
28         else:
29             print(f"X FAIL: baseline_results should contain dicts with 'accuracy' and 'failures' keys")
30             checks_failed += 1
31             break
32
33     # Check best baseline variables
34     if 'best_baseline_prompt' in globals() and 'best_baseline_accuracy' in globals():
35         print("✓ PASS: best_baseline_prompt and best_baseline_accuracy variables exist")
36         checks_passed += 1
37
38     # Check accuracy is valid
39     if isinstance(best_baseline_accuracy, (int, float)) and 0.0 <= best_baseline_accuracy <= 1.0:

```

```

40         print(f"✓ PASS: Best baseline accuracy is valid: {best_baseline_accuracy:.2%}")
41         checks_passed += 1
42
43         if best_baseline_accuracy > 0:
44             print("✓ PASS: Best baseline accuracy > 0% (model is working)")
45             checks_passed += 1
46         else:
47             print("⚠ WARNING: Best baseline accuracy is 0% - model may not be working correctly")
48     else:
49         print(f"X FAIL: best_baseline_accuracy should be between 0.0 and 1.0, got {best_baseline_accuracy}")
50         checks_failed += 1
51     else:
52         print("X FAIL: best_baseline_prompt and/or best_baseline_accuracy not defined")
53         checks_failed += 1
54     else:
55         print("X FAIL: baseline_results variable not found. Did you run the baseline evaluation cell?")
56         checks_failed += 1
57 except Exception as e:
58     print(f"X FAIL: Error checking baseline results: {e}")
59     checks_failed += 1
60
61 print("\n" + "="*70)
62 if checks_failed == 0:
63     print(f"All checks passed! ✓ ({checks_passed} checks)")
64 else:
65     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
66     print("Please review your implementation and try again.")
67 print("="*70)
68

```

```

=====
VALIDATING: Baseline evaluation results
=====
✓ PASS: baseline_results variable exists
✓ PASS: At least 3 baseline prompts tested (5 prompts)
✓ PASS: best_baseline_prompt and best_baseline_accuracy variables exist
✓ PASS: Best baseline accuracy is valid: 70.00%
✓ PASS: Best baseline accuracy > 0% (model is working)

=====
All checks passed! ✓ (6 checks)
=====

```

✓ TASK 2: Generative Optimization Loop (7 points)

Now we'll build the core APE system: a feedback loop where the model analyzes its errors and generates improved prompts.

2.1: Error Analysis Prompt — Construct a meta-prompt that shows the model its failures

2.2: Candidate Generation — Generate multiple candidate prompts

2.3: Optimization Loop — Implement the full iterative optimization cycle

✓ Task 2.1: Error Analysis Meta-Prompt

Goal: Create a "meta-prompt" that instructs the model to analyze its errors and suggest an improved prompt.

What is a meta-prompt? A meta-prompt is a prompt that asks the model to generate another prompt. We'll show the model its failures and ask it to write a better instruction prompt.

Requirements:

- Randomly sample `n_examples` from the `failed_cases` list
- Format each failed case to show: the question, the model's prediction, and the correct answer
- Construct a meta-prompt string that:
 - Explains the context (using an AI to solve math word problems)
 - Shows the current prompt being used
 - Presents the failed examples in a clear format
 - Asks the model to write a better prompt
 - **Important:** Emphasize that the new prompt MUST contain `{question}` as a placeholder
 - Instruct the model to output ONLY the new prompt, nothing else

Example meta-prompt structure:

```

I am using an AI to solve math word problems. The current instruction prompt is:

"{current_prompt}"

```

However, the AI made mistakes on these examples:

Example 1:

Question: [question]

Model predicted: [prediction]

Correct answer: [target]

...

Write a better instruction prompt that would help the AI answer correctly.

The prompt must contain {question} as a placeholder.

Output only the new prompt, nothing else.

New prompt:

Success criteria:

- Meta-prompt contains the current prompt
- Meta-prompt contains at least one failed example
- Meta-prompt asks for an improved prompt
- Meta-prompt explicitly mentions the `{question}` placeholder requirement

```

1 def create_meta_prompt(
2     current_prompt: str,
3     failed_cases: List[Dict],
4     n_examples: int = 3
5 ) -> str:
6     """
7     Create a meta-prompt for the model to analyze errors and suggest a better prompt.
8
9     Args:
10         current_prompt: The prompt template currently being used
11         failed_cases: List of failed examples with 'question', 'prediction', 'target'
12         n_examples: Number of failed examples to include
13
14     Returns:
15         A meta-prompt string asking the model to improve the prompt
16     """
17     ## YOUR_CODE_STARTS_HERE
18     n = min(n_examples, len(failed_cases))
19     samples = failed_cases[:n] # first n failures
20
21     meta_string = [] # prompt elements stored
22     meta_string.append("I am using an AI to solve math word problems. The current instruction prompt is:")
23     meta_string.append(f"\n{current_prompt}\n")
24     meta_string.append("However, the AI made mistakes on these examples:")
25     for i, exp in enumerate(samples, start = 1):
26         meta_string.append(f"\nExample {i}:")
27         meta_string.append(f"Question: {exp['question']}\n")
28         meta_string.append(f"Predicted: {exp['prediction']}\n")
29         meta_string.append(f"Correct: {exp['target']}\n")
30     meta_string.append(
31         "\nWrite a better instruction prompt that would help the AI answer correctly.\n"
32         "The prompt must contain {question} as a placeholder.\n"
33         "Output only the new prompt, nothing else.\n\n"
34         "New prompt:"
35     )
36     meta_prompt = "\n".join(meta_string)
37
38     return meta_prompt
39     ## YOUR_CODE_ENDS_HERE
40

```

```

1 # Answer Checkpoint: Task 2.1 - Meta-Prompt Creation
2 print("="*70)
3 print("VALIDATING: create_meta_prompt function")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 try:
10     # Test with sample data
11     if 'best_baseline_prompt' in globals() and 'best_baseline_failures' in globals():
12         test_meta = create_meta_prompt(best_baseline_prompt, best_baseline_failures[:5], n_examples=2)
13
14     # Check 1: Returns a string
15     if isinstance(test_meta, str):

```

```

16         print("✓ PASS: Function returns a string")
17         checks_passed += 1
18     else:
19         print(f"X FAIL: Function should return string, got {type(test_meta)}")
20         checks_failed += 1
21
22     # Check 2: Contains current prompt
23     if best_baseline_prompt in test_meta or best_baseline_prompt.replace('\n', ' ') in test_meta:
24         print("✓ PASS: Meta-prompt contains current prompt")
25         checks_passed += 1
26     else:
27         print("X FAIL: Meta-prompt should contain the current prompt")
28         checks_failed += 1
29
30     # Check 3: Contains failed examples      #?
31     if len(best_baseline_failures) > 0:
32         first_failure_q = best_baseline_failures[0].get('question', '')
33         if first_failure_q and first_failure_q in test_meta:
34             print("✓ PASS: Meta-prompt contains failed examples")
35             checks_passed += 1
36         else:
37             print("⚠ WARNING: Meta-prompt may not contain failed examples")
38
39     # Check 4: Mentions {question} placeholder requirement
40     if '{question}' in test_meta or 'question' in test_meta.lower():
41         print("✓ PASS: Meta-prompt mentions question placeholder")
42         checks_passed += 1
43     else:
44         print("⚠ WARNING: Meta-prompt should mention {question} placeholder requirement")
45
46     # Check 5: Asks for improved prompt
47     if any(word in test_meta.lower() for word in ['better', 'improve', 'new prompt', 'write']):
48         print("✓ PASS: Meta-prompt asks for improved prompt")
49         checks_passed += 1
50     else:
51         print("⚠ WARNING: Meta-prompt should ask for an improved prompt")
52
53     else:
54         print("⚠ SKIP: best_baseline_prompt or best_baseline_failures not found. Run Task 1.3 first.")
55 except Exception as e:
56     print(f"X FAIL: Error testing create_meta_prompt: {e}")
57     checks_failed += 1
58
59 print("\n" + "="*70)
60 if checks_failed == 0:
61     print(f"All checks passed! ✓ ({checks_passed} checks)")
62 else:
63     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
64 print("="*70)

```

```

=====
VALIDATING: create_meta_prompt function
=====
✓ PASS: Function returns a string
✓ PASS: Meta-prompt contains current prompt
✓ PASS: Meta-prompt contains failed examples
✓ PASS: Meta-prompt mentions question placeholder
✓ PASS: Meta-prompt asks for improved prompt

=====
All checks passed! ✓ (5 checks)
=====

```

```

1 # Test the meta-prompt creation
2 sample_meta_prompt = create_meta_prompt(best_baseline_prompt, best_baseline_failures, n_examples=2)
3 print("Sample Meta-Prompt:")
4 print("=" * 70)
5 print(sample_meta_prompt)
6 print("=" * 70)
7

```

Sample Meta-Prompt:

```

=====
I am using an AI to solve math word problems. The current instruction prompt is:
"{question} answer"

However, the AI made mistakes on these examples:

Example 1:
Question: Alice has 10 apples. Bob gives her 36 more apples. How many apples does Alice have now?
Predicted: 40
Correct: 46

Example 2:

```

Question: There are 26 birds on a tree. 17 birds fly away. How many birds are left?

Predicted: 19

Correct: 9

Write a better instruction prompt that would help the AI answer correctly.

The prompt must contain {question} as a placeholder.

Output only the new prompt, nothing else.

New prompt:

=====

Task 2.2: Candidate Prompt Generation

Goal: Generate multiple diverse candidate prompts from the meta-prompt using sampling.

Step-by-step instructions:

- 1. Tokenize the meta-prompt:** Convert the meta-prompt string to token IDs, similar to `generate_response()`
- 2. Generate multiple candidates:** Loop to generate `n_candidates` valid prompts:
 - Use `model.generate()` with `do_sample=True` for diversity
 - Set `temperature` (e.g., 0.7) - higher = more diverse, lower = more focused
 - Use `top_p=0.9` for nucleus sampling (helps with quality)
 - Generate extra candidates (e.g., `n_candidates * 5`) since some may be invalid
- 3. Clean and normalize candidates:**
 - Decode only the newly generated tokens (not the input)
 - Strip whitespace
 - Remove surrounding quotes if present (`"..."` or `'...'`)
 - Normalize placeholder: replace variations like `{example_question}` or `{the_question}` with `{question}`
 - If candidate mentions "question" but has no placeholder, try to add `{question}`
- 4. Validate candidates:**
 - Must contain `{question}` placeholder
 - Must be reasonable length (< 200 characters)
 - Must be able to be formatted: `candidate.format(question="test")` should work
 - Should not be duplicate of existing candidates
 - Should be meaningfully different from current prompt (optional but helpful)
- 5. Return valid candidates:** Return up to `n_candidates` valid prompts

Key parameters:

- `temperature`: Controls randomness (0.7-0.9 good for diversity)
- `top_p`: Nucleus sampling threshold (0.9 is standard)
- `max_new_tokens`: Limit generation length (150-200 tokens recommended for complete prompts)

Success criteria:

- Function returns a list of strings
- All candidates contain `{question}` placeholder
- All candidates can be formatted without `KeyError`
- At least 1 candidate generated (even if fewer than requested)

```

1 def generate_candidate_prompts(
2     model,
3     tokenizer,
4     meta_prompt: str,
5     n_candidates: int = 5,
6     temperature: float = 0.7
7 ) -> List[str]:
8     """
9     Generate multiple candidate prompts using the meta-prompt.
10
11     Args:
12         model: The language model
13         tokenizer: The tokenizer
14         meta_prompt: The meta-prompt asking for improved prompts
15         n_candidates: Number of candidates to generate
16         temperature: Sampling temperature for diversity
17
18     Returns:
19         List of valid candidate prompt strings (containing {question})
20     """
21     ## YOUR_CODE_STARTS_HERE

```

```

22 gen_n = n_candidates * 5
23 tok_prompt = tokenizer(meta_prompt, return_tensors = "pt", padding = False).to(device)
24 valid_prompts = []
25
26 if tokenizer.pad_token_id is not None:
27     pad_token_id = tokenizer.pad_token_id
28 else:
29     pad_token_id = tokenizer.eos_token_id
30
31 with torch.no_grad():
32     outputs = model.generate(
33         **tok_prompt,
34         max_new_tokens = 150,
35         do_sample = True,
36         temperature = temperature,
37         top_k = 10,
38         num_return_sequences = gen_n,
39         pad_token_id = pad_token_id,
40         return_dict_in_generate = True
41     )
42
43 prompt_len = tok_prompt["input_ids"].shape[1]
44 past_gen = set()
45 for seq in outputs.sequences:
46     gen_ids = seq[prompt_len:] # only generated tokens extracted
47     generated = tokenizer.decode(gen_ids, skip_special_tokens = True).strip()
48     if not generated:
49         continue
50
51     if (generated.startswith('"') and generated.endswith('"')) or (generated.startswith("'") and generated.endswith('
52         generated = generated[1:-1].strip() # remove sorrounding quotes
53
54     #normalize placeholders
55     generated = re.sub(r"\{[^}]*question[^}]*\}", "{question}", generated, flags = re.IGNORECASE)
56
57     if "{question}" not in generated and "question" in generated.lower():
58         generated = generated.rstrip(". ")
59         generated += "\n\nQuestion: {question}" # add question if no placeholder
60     if "{question}" not in generated:
61         continue
62
63     if len(generated) > 150:
64         continue
65     try: # able to be formatted
66         _ = generated.format(question = "test")
67     except Exception:
68         continue
69
70     if generated.strip().lower() in past_gen:
71         continue # if it already exists
72     past_gen.add(generated.strip().lower())
73     valid_prompts.append(generated)
74     if len(valid_prompts) >= n_candidates:
75         break
76
77 return valid_prompts
78
79 ## YOUR_CODE_ENDS_HERE
80

```

```

1 # Answer Checkpoint: Task 2.2 - Candidate Generation
2 print("="*70)
3 print("VALIDATING: generate_candidate_prompts function")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 try:
10     if 'sample_meta_prompt' in globals():
11         test_candidates = generate_candidate_prompts(model, tokenizer, sample_meta_prompt, n_candidates=3, temperature=
12
13         # Check 1: Returns a list
14         if isinstance(test_candidates, list):
15             print("✓ PASS: Function returns a list")
16             checks_passed += 1
17         else:
18             print(f"X FAIL: Function should return list, got {type(test_candidates)}")
19             checks_failed += 1
20
21         # Check 2: All items are strings
22         if all(isinstance(c, str) for c in test_candidates):

```

```

23     print("✓ PASS: All candidates are strings")
24     checks_passed += 1
25     else:
26         print("✗ FAIL: All candidates should be strings")
27         checks_failed += 1
28
29     # Check 3: All contain {question}
30     if len(test_candidates) > 0:
31         all_have_placeholder = all('{question}' in c for c in test_candidates)
32         if all_have_placeholder:
33             print("✓ PASS: All candidates contain {question} placeholder")
34             checks_passed += 1
35         else:
36             missing = [i for i, c in enumerate(test_candidates) if '{question}' not in c]
37             print(f"✗ FAIL: Candidates {missing} missing {{question}} placeholder")
38             checks_failed += 1
39
40     # Check 4: All can be formatted
41     format_errors = []
42     for i, c in enumerate(test_candidates):
43         try:
44             c.format(question="test")
45         except KeyError as e:
46             format_errors.append(i)
47
48     if len(format_errors) == 0:
49         print("✓ PASS: All candidates can be formatted")
50         checks_passed += 1
51     else:
52         print(f"✗ FAIL: Candidates {format_errors} cannot be formatted (KeyError)")
53         checks_failed += 1
54
55     # Check 5: At least 1 candidate
56     if len(test_candidates) >= 1:
57         print(f"✓ PASS: Generated at least 1 candidate ({len(test_candidates)} candidates)")
58         checks_passed += 1
59     else:
60         print("⚠ WARNING: No candidates generated (this may be okay if model is struggling)")
61     else:
62         print("⚠ WARNING: No candidates generated")
63     else:
64         print("⚠ SKIP: sample_meta_prompt not found. Run Task 2.1 test cell first.")
65 except Exception as e:
66     print(f"✗ FAIL: Error testing generate_candidate_prompts: {e}")
67     import traceback
68     traceback.print_exc()
69     checks_failed += 1
70
71 print("\n" + "="*70)
72 if checks_failed == 0:
73     print(f"All checks passed! ✓ ({checks_passed} checks)")
74 else:
75     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
76 print("="*70)
77

```

```

=====
VALIDATING: generate_candidate_prompts function
=====
✓ PASS: Function returns a list
✓ PASS: All candidates are strings
✓ PASS: All candidates contain {question} placeholder
✓ PASS: All candidates can be formatted
✓ PASS: Generated at least 1 candidate (3 candidates)

=====
All checks passed! ✓ (5 checks)
=====

```

```

1 # Test candidate generation
2 print("Generating candidate prompts...")
3 candidates = generate_candidate_prompts(model, tokenizer, sample_meta_prompt, n_candidates=3)
4 print(f"\nGenerated {len(candidates)} valid candidates:")
5 for i, c in enumerate(candidates, 1):
6     print(f" {i}. {c}")
7

```

Generating candidate prompts...

Generated 3 valid candidates:

1. Give me the updated answer to {question}
2. Give the answer to the question posed in the first example

Question: {question}

3. Given the following situation: {question}, what is the updated answer?

Task 2.3: The Full Optimization Loop

Goal: Implement the complete Automatic Prompt Engineering (APE) algorithm that iteratively improves prompts.

Algorithm Overview:

The APE algorithm follows this iterative process:

1. **Evaluate** current prompt on training and validation data
2. **Analyze** errors to understand failure modes
3. **Generate** candidate improved prompts using meta-prompting
4. **Evaluate** candidates on validation data
5. **Select** best candidate (if better than current)
6. **Repeat** until convergence or max iterations

Requirements:

- **Initialize history:** Create a dictionary to track iterations, prompts tried, accuracies, best prompt, and best accuracy
- **Main optimization loop** (for each iteration):
 - Evaluate current prompt on both training data (for error analysis) and validation data (for selection)
 - Update history with current iteration results
 - Check stopping conditions (target accuracy reached, no failures, max iterations, no valid candidates)
 - Generate candidate prompts using meta-prompting from training failures
 - Evaluate all candidates on validation data and track the best one
 - Update prompt if best candidate is better than current
- **Return history:** Dictionary with all optimization information

Stopping conditions:

- Target accuracy reached (e.g., 95%)
- No failures to analyze (perfect accuracy on training)
- Maximum iterations reached
- No valid candidates generated

Success criteria:

- History dictionary has correct structure with all required keys
- At least 1 iteration completed
- Best accuracy $\geq$ initial accuracy (or explanation if not improved)
- Prompts list contains at least the initial prompt

```

1 def optimize_prompt(
2     model,
3     tokenizer,
4     initial_prompt: str,
5     train_data: List[Dict],
6     val_data: List[Dict],
7     n_iterations: int = 3,
8     n_candidates: int = 3,
9     target_accuracy: float = 0.95,
10    verbose: bool = True
11 ) -> Dict:
12     """
13     Run the full prompt optimization loop.
14
15     Args:
16         model: The language model
17         tokenizer: The tokenizer
18         initial_prompt: Starting prompt template
19         train_data: Data for error analysis
20         val_data: Data for evaluation
21         n_iterations: Maximum optimization iterations
22         n_candidates: Candidates to generate per iteration
23         target_accuracy: Stop if this accuracy is reached
24         verbose: Print progress
25
26     Returns:
27         Dictionary with optimization history and best prompt
28     """
29     ## YOUR_CODE_STARTS_HERE
30     history = {
31         "iterations": [],

```

```

32     "prompts": [],
33     "best_prompt": initial_prompt,
34     "accuracies": [],
35     "best_accuracy": 0.0
36 }
37
38 curr_prompt = initial_prompt
39 for i in range(1, n_iterations + 1):
40     train_acc, train_fail = evaluate_prompt(model, tokenizer, curr_prompt, train_data, verbose = verbose)
41     val_acc, _ = evaluate_prompt(model, tokenizer, curr_prompt, val_data, verbose = verbose)
42     history["iterations"].append({
43         "prompt": curr_prompt,
44         "train_accuracy": train_acc,
45         "val_accuracy": val_acc,
46         "n_failures": len(train_fail)
47     })
48     history["prompts"].append(curr_prompt)
49     history["accuracies"].append(val_acc)
50
51     if val_acc > history["best_accuracy"]:
52         history["best_accuracy"] = val_acc
53         history["best_prompt"] = curr_prompt
54
55     if val_acc >= target_accuracy or not train_fail:
56         break
57
58     meta_prompt = create_meta_prompt(curr_prompt, train_fail, n_examples=3)
59     candidates = generate_candidate_prompts(model, tokenizer, meta_prompt, n_candidates=n_candidates)
60     if not candidates:
61         break
62
63     best_cand = None
64     best_cand_acc = 0.0
65     for cand in candidates:
66         acc, _ = evaluate_prompt(model, tokenizer, cand, train_data)
67         if acc > best_cand_acc:
68             best_cand_acc = acc
69             best_cand = cand
70
71     if best_cand_acc > val_acc:
72         curr_prompt = best_cand
73     else:
74         break
75
76 return history
77 ## YOUR_CODE_ENDS_HERE
78

```

```

1 # Run the optimization loop
2 print("Starting Automatic Prompt Optimization...")
3 print(f"Initial prompt: '{best_baseline_prompt}'")
4 print(f"Initial accuracy: {best_baseline_accuracy:.2%}")
5
6 optimization_result = optimize_prompt(
7     model=model,
8     tokenizer=tokenizer,
9     initial_prompt=best_baseline_prompt,
10    train_data=train_data,
11    val_data=val_data,
12    n_iterations=3,
13    n_candidates=3,
14    target_accuracy=0.95,
15    verbose=True
16 )
17
18 print("\n" + "="*70)
19 print("OPTIMIZATION COMPLETE")
20 print("="*70)
21 print(f"Best prompt found: '{optimization_result['best_prompt']}'")
22 print(f"Best accuracy: {optimization_result['best_accuracy']:.2%}")
23

```

Starting Automatic Prompt Optimization...
Initial prompt: '{question} answer'
Initial accuracy: 70.00%

=====

OPTIMIZATION COMPLETE

=====

Best prompt found: '"{question}" - Correct answer'

Best accuracy: 85.00%

```

1 # Answer Checkpoint: Task 2.3 - Optimization Loop
2 print("="*70)
3 print("VALIDATING: optimize_prompt function")
4 print("="*70)
5
6 checks_passed = 0
7 checks_failed = 0
8
9 try:
10     if 'optimization_result' in globals():
11         result = optimization_result
12
13         # Check 1: Returns a dictionary
14         if isinstance(result, dict):
15             print("✓ PASS: Function returns a dictionary")
16             checks_passed += 1
17         else:
18             print(f"X FAIL: Function should return dict, got {type(result)}")
19             checks_failed += 1
20
21         # Check 2: Has required keys
22         required_keys = ['iterations', 'prompts', 'accuracies', 'best_prompt', 'best_accuracy']
23         missing_keys = [k for k in required_keys if k not in result]
24         if len(missing_keys) == 0:
25             print("✓ PASS: History dictionary has all required keys")
26             checks_passed += 1
27         else:
28             print(f"X FAIL: Missing keys: {missing_keys}")
29             checks_failed += 1
30
31         # Check 3: At least 1 iteration completed
32         if len(result.get('iterations', [])) >= 1:
33             print(f"✓ PASS: At least 1 iteration completed ({len(result['iterations'])} iterations)")
34             checks_passed += 1
35         else:
36             print("X FAIL: Should complete at least 1 iteration")
37             checks_failed += 1
38
39         # Check 4: Prompts list contains initial prompt
40         if len(result.get('prompts', [])) >= 1:
41             print("✓ PASS: Prompts list contains at least initial prompt")
42             checks_passed += 1
43         else:
44             print("X FAIL: Prompts list should contain at least initial prompt")
45             checks_failed += 1
46
47         # Check 5: Best accuracy is valid
48         best_acc = result.get('best_accuracy', -1)
49         if isinstance(best_acc, (int, float)) and 0.0 <= best_acc <= 1.0:
50             print(f"✓ PASS: Best accuracy is valid: {best_acc:.2%}")
51             checks_passed += 1
52         # Check if improved (or at least maintained)
53         if 'best_baseline_accuracy' in globals():
54             if best_acc >= best_baseline_accuracy:
55                 print(f"✓ PASS: Best accuracy ({best_acc:.2%}) >= baseline ({best_baseline_accuracy:.2%})")
56                 checks_passed += 1
57             else:
58                 print(f"△ INFO: Best accuracy ({best_acc:.2%}) < baseline ({best_baseline_accuracy:.2%})")
59                 print("   This is okay - optimization doesn't always improve immediately")
60         else:
61             print(f"X FAIL: best_accuracy should be between 0.0 and 1.0, got {best_acc}")
62             checks_failed += 1
63
64         # Check 6: Iterations structure
65         if len(result.get('iterations', [])) > 0:
66             first_iter = result['iterations'][0]
67             iter_keys = ['prompt', 'train_accuracy', 'val_accuracy', 'n_failures']
68             if all(k in first_iter for k in iter_keys):
69                 print("✓ PASS: Iteration data has correct structure")
70                 checks_passed += 1
71             else:
72                 missing = [k for k in iter_keys if k not in first_iter]
73                 print(f"△ WARNING: Iteration data missing keys: {missing}")
74         else:
75             print("△ SKIP: optimization_result not found. Run the optimization loop cell first.")
76 except Exception as e:
77     print(f"X FAIL: Error testing optimize_prompt: {e}")
78     import traceback
79     traceback.print_exc()
80     checks_failed += 1
81
82 print("\n" + "="*70)

```

```

83 if checks_failed == 0:
84     print(f"All checks passed! ✓ ({checks_passed} checks)")
85 else:
86     print(f"Some checks failed: {checks_passed} passed, {checks_failed} failed")
87 print("=*70)
88

```

```

=====
VALIDATING: optimize_prompt function
=====
✓ PASS: Function returns a dictionary
✓ PASS: History dictionary has all required keys
✓ PASS: At least 1 iteration completed (3 iterations)
✓ PASS: Prompts list contains at least initial prompt
✓ PASS: Best accuracy is valid: 85.00%
✓ PASS: Best accuracy (85.00%) >= baseline (70.00%)
✓ PASS: Iteration data has correct structure

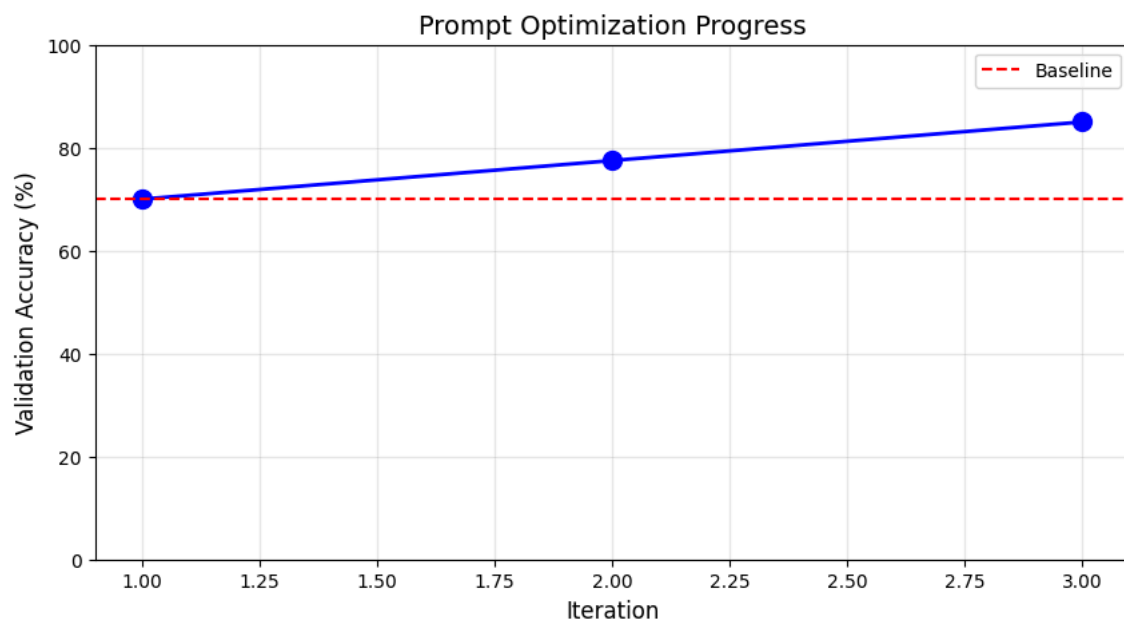
=====
All checks passed! ✓ (7 checks)
=====

```

```

1 # Visualize optimization progress
2 plt.figure(figsize=(10, 5))
3 plt.plot(range(1, len(optimization_result['accuracies']) + 1),
4          [a * 100 for a in optimization_result['accuracies']],
5          'bo-', linewidth=2, markersize=10)
6 plt.axhline(y=best_baseline_accuracy * 100, color='r', linestyle='--', label='Baseline')
7 plt.xlabel('Iteration', fontsize=12)
8 plt.ylabel('Validation Accuracy (%)', fontsize=12)
9 plt.title('Prompt Optimization Progress', fontsize=14)
10 plt.legend()
11 plt.grid(True, alpha=0.3)
12 plt.ylim(0, 100)
13 plt.show()
14

```



```

1 # Final evaluation on held-out test set
2 print("Final Evaluation on Test Set")
3 print("=*70)
4
5 # Baseline on test
6 baseline_test_acc, baseline_test_failures = evaluate_prompt(
7     model, tokenizer, best_baseline_prompt, test_data
8 )
9 print(f"Baseline prompt: '{best_baseline_prompt}'")
10 print(f"Baseline test accuracy: {baseline_test_acc:.2%}")
11
12 print()
13
14 # Optimized on test
15 optimized_prompt = optimization_result['best_prompt']
16 optimized_test_acc, optimized_test_failures = evaluate_prompt(
17     model, tokenizer, optimized_prompt, test_data
18 )
19 print(f"Optimized prompt: '{optimized_prompt}'")

```

```

20 print(f"Optimized test accuracy: {optimized_test_acc:.2%}")
21
22 print()
23 improvement = (optimized_test_acc - baseline_test_acc) * 100
24 print(f"Improvement: {improvement:+.1f} percentage points")
25

```

Final Evaluation on Test Set

=====

Baseline prompt: '{question} answer'

Baseline test accuracy: 75.00%

Optimized prompt: '"{question}" - Correct answer'

Optimized test accuracy: 95.00%

Improvement: +20.0 percentage points

✓ TASK 3: Interpretability Analysis (6 points)

Now we'll analyze *why* the optimized prompt performs better using white-box techniques.

3.1: Confidence Analysis — Compare model confidence (logit values) between prompts

3.2: Attention Analysis — Visualize and compare attention patterns

✓ Task 3.1: Confidence Analysis

Compare how "confident" the model is when using the baseline vs optimized prompt:

1. Extract logits for the generated answer tokens
2. Convert to probabilities
3. Compare the probability assigned to correct tokens

```

1 def analyze_confidence(
2     model,
3     tokenizer,
4     prompt_template: str,
5     question: str,
6     target_answer: str
7 ) -> Dict:
8     """
9     Analyze model confidence for a given prompt and question.
10
11     Args:
12         model: The language model
13         tokenizer: The tokenizer
14         prompt_template: The prompt template to use
15         question: The question to answer
16         target_answer: The correct answer
17
18     Returns:
19         Dictionary with confidence metrics
20     """
21     ## YOUR_CODE_STARTS_HERE
22     result = generate_response(
23         model = model,
24         tokenizer = tokenizer,
25         prompt_template = prompt_template,
26         question = question,
27         max_new_tokens = 150,
28         return_logits = True
29     )
30
31     response = result["response"]
32     logits_list = result.get("logits", None)
33     target_tokens = tokenizer(target_answer, add_special_tokens = False)["input_ids"]
34
35     if logits_list is None or len(logits_list) == 0:
36         return{
37             "response": response,
38             "first_token_confidence": 0.0,
39             "entropy": 0.0
40         }
41
42     first_tok_id = target_tokens[0]
43     first_logits = logits_list[0]
44     if first_logits.dim() == 1:
45         probs = F.softmax(first_logits, dim = -1)
46     else:

```

```

47     probs = F.softmax(first_logits.squeeze(0), dim = -1)
48
49     first_tok_conf = probs[first_tok_id].item()
50     entropy = -torch.sum(probs * torch.log(probs + 1e-12)).item()
51
52     return{
53         "response": response,
54         "first_token_confidence": first_tok_conf,
55         "entropy": entropy
56     }
57     ## YOUR_CODE_ENDS_HERE
58

```

```

1 # Compare confidence between baseline and optimized prompts
2 print("Confidence Analysis: Baseline vs Optimized Prompt")
3 print("="*70)
4
5 # Select examples to analyze
6 analysis_examples = test_data[:10]
7
8 baseline_confidences = []
9 optimized_confidences = []
10 baseline_entropies = []
11 optimized_entropies = []
12
13 for example in analysis_examples:
14     question = example['question']
15     target = example['answer']
16
17     # Analyze with baseline prompt
18     baseline_result = analyze_confidence(model, tokenizer, best_baseline_prompt, question, target)
19     baseline_confidences.append(baseline_result['first_token_confidence'])
20     baseline_entropies.append(baseline_result['entropy'])
21
22     # Analyze with optimized prompt
23     optimized_result = analyze_confidence(model, tokenizer, optimized_prompt, question, target)
24     optimized_confidences.append(optimized_result['first_token_confidence'])
25     optimized_entropies.append(optimized_result['entropy'])
26
27 print(f"Average First Token Confidence:")
28 print(f"   Baseline: {np.mean(baseline_confidences):.4f}")
29 print(f"   Optimized: {np.mean(optimized_confidences):.4f}")
30 print()
31 print(f"Average Entropy (lower = more confident):")
32 print(f"   Baseline: {np.mean(baseline_entropies):.4f}")
33 print(f"   Optimized: {np.mean(optimized_entropies):.4f}")
34

```

```

Confidence Analysis: Baseline vs Optimized Prompt
=====
Average First Token Confidence:
Baseline:  0.0000
Optimized: 0.0000

Average Entropy (lower = more confident):
Baseline:  1.4999
Optimized: 0.4897

```

```

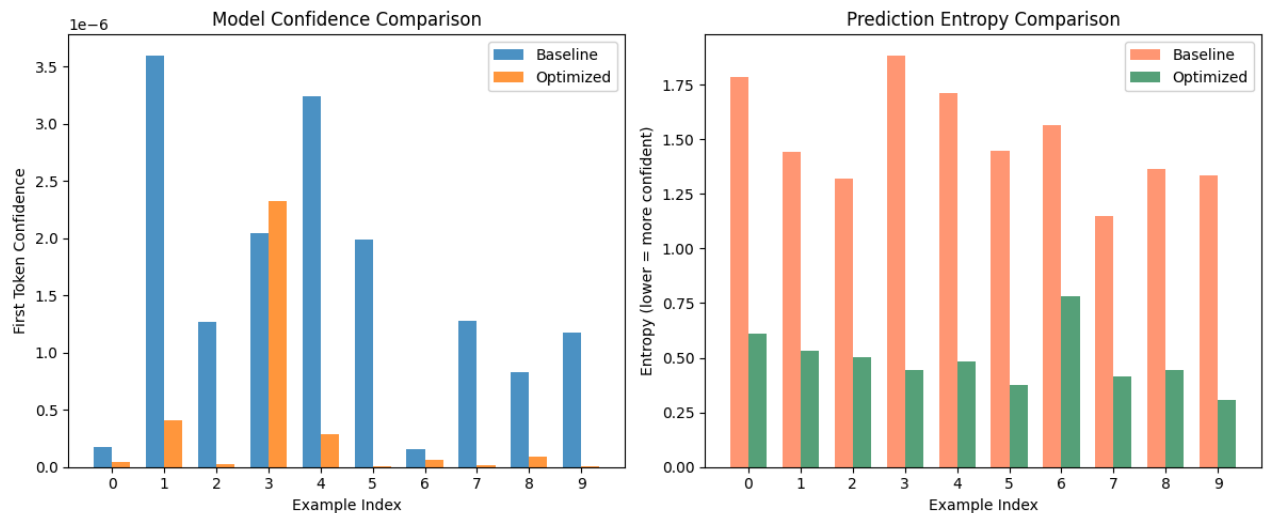
1 # Visualize confidence comparison
2 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
3
4 # Confidence bar chart
5 ax1 = axes[0]
6 x = np.arange(len(analysis_examples))
7 width = 0.35
8 ax1.bar(x - width/2, baseline_confidences, width, label='Baseline', alpha=0.8)
9 ax1.bar(x + width/2, optimized_confidences, width, label='Optimized', alpha=0.8)
10 ax1.set_xlabel('Example Index')
11 ax1.set_ylabel('First Token Confidence')
12 ax1.set_title('Model Confidence Comparison')
13 ax1.legend()
14 ax1.set_xticks(x)
15
16 # Entropy comparison
17 ax2 = axes[1]
18 ax2.bar(x - width/2, baseline_entropies, width, label='Baseline', alpha=0.8, color='coral')
19 ax2.bar(x + width/2, optimized_entropies, width, label='Optimized', alpha=0.8, color='seagreen')
20 ax2.set_xlabel('Example Index')
21 ax2.set_ylabel('Entropy (lower = more confident)')
22 ax2.set_title('Prediction Entropy Comparison')
23 ax2.legend()
24 ax2.set_xticks(x)

```

```

25
26 plt.tight_layout()
27 plt.show()
28

```



Task 3.2: Attention Analysis

Visualize attention patterns to understand what the model "looks at" with different prompts:

1. Extract attention weights from the model (decoder attention for decoder-only models)
2. Create attention heatmaps
3. Compare attention concentration between prompts

```

1 def get_encoder_attention(
2     model,
3     tokenizer,
4     prompt_template: str,
5     question: str
6 ) -> Tuple[torch.Tensor, List[str]]:
7     """
8     Extract attention weights for analysis (decoder attention for decoder-only models).
9
10    Args:
11        model: The language model
12        tokenizer: The tokenizer
13        prompt_template: The prompt template
14        question: The question
15
16    Returns:
17        Tuple of (attention_weights, token_labels)
18        attention_weights: Tensor of shape (n_layers, n_heads, seq_len, seq_len)
19        token_labels: List of token strings for labeling
20    """
21    ## YOUR_CODE_STARTS_HERE
22    prompt = prompt_template.format(question = question)
23    inputs = tokenizer(prompt, return_tensors = "pt").to(model.device)
24
25    with torch.no_grad():
26        outputs = model(
27            **inputs,
28            output_attentions = True,
29            use_cache = False
30        )
31
32    attentions = torch.stack(outputs.attentions)
33    attentions = attentions.squeeze(1)
34    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
35
36    return attentions, tokens
37
38    ## YOUR_CODE_ENDS_HERE
39

```

```

1 def visualize_attention(
2     attention: torch.Tensor,
3     tokens: List[str],
4     layer: int = -1,
5     head: int = 0,
6     title: str = "Attention Heatmap"
7 ):
8     """
9     Visualize attention weights as a heatmap.
10
11     Args:
12         attention: Tensor of shape (n_layers, n_heads, seq_len, seq_len)
13         tokens: List of token strings
14         layer: Which layer to visualize (-1 for last)
15         head: Which attention head to visualize
16         title: Plot title
17     """
18     ## YOUR_CODE_STARTS_HERE
19
20     att = attention[layer, head].cpu().numpy()
21     fig, ax = plt.subplots(figsize = (8,8))
22     im = ax.imshow(att, cmap = "viridis")
23
24     ax.set_xticks(range(len(tokens)))
25     ax.set_yticks(range(len(tokens)))
26     ax.set_xticklabels(tokens, rotation = 90)
27     ax.set_yticklabels(tokens)
28
29     ax.set_xlabel("Key tokens")
30     ax.set_ylabel("Query tokens")
31     ax.set_title(title)
32
33     fig.colorbar(im, ax = ax, fraction = 0.045, pad = 0.05)
34
35     plt.tight_layout()
36     plt.show()
37     ## YOUR_CODE_ENDS_HERE
38

```

```

1 # Compare attention patterns for a specific example
2 example = test_data[0]
3 print(f"Analyzing attention for question:")
4 print(f"   {example['question']}")
5 print(f"   Answer: {example['answer']}")
6 print()
7
8 # Get attention for baseline prompt
9 baseline_attn, baseline_tokens = get_encoder_attention(
10     model, tokenizer, best_baseline_prompt, example['question']
11 )
12 print(f"Baseline prompt: '{best_baseline_prompt}'")
13 visualize_attention(
14     baseline_attn, baseline_tokens,
15     layer=-1, head=0,
16     title="Baseline Prompt - Last Layer, Head 0"
17 )
18
19 # Get attention for optimized prompt
20 optimized_attn, optimized_tokens = get_encoder_attention(
21     model, tokenizer, optimized_prompt, example['question']
22 )
23 print(f"Optimized prompt: '{optimized_prompt}'")
24 visualize_attention(
25     optimized_attn, optimized_tokens,
26     layer=-1, head=0,
27     title="Optimized Prompt - Last Layer, Head 0"
28 )
29

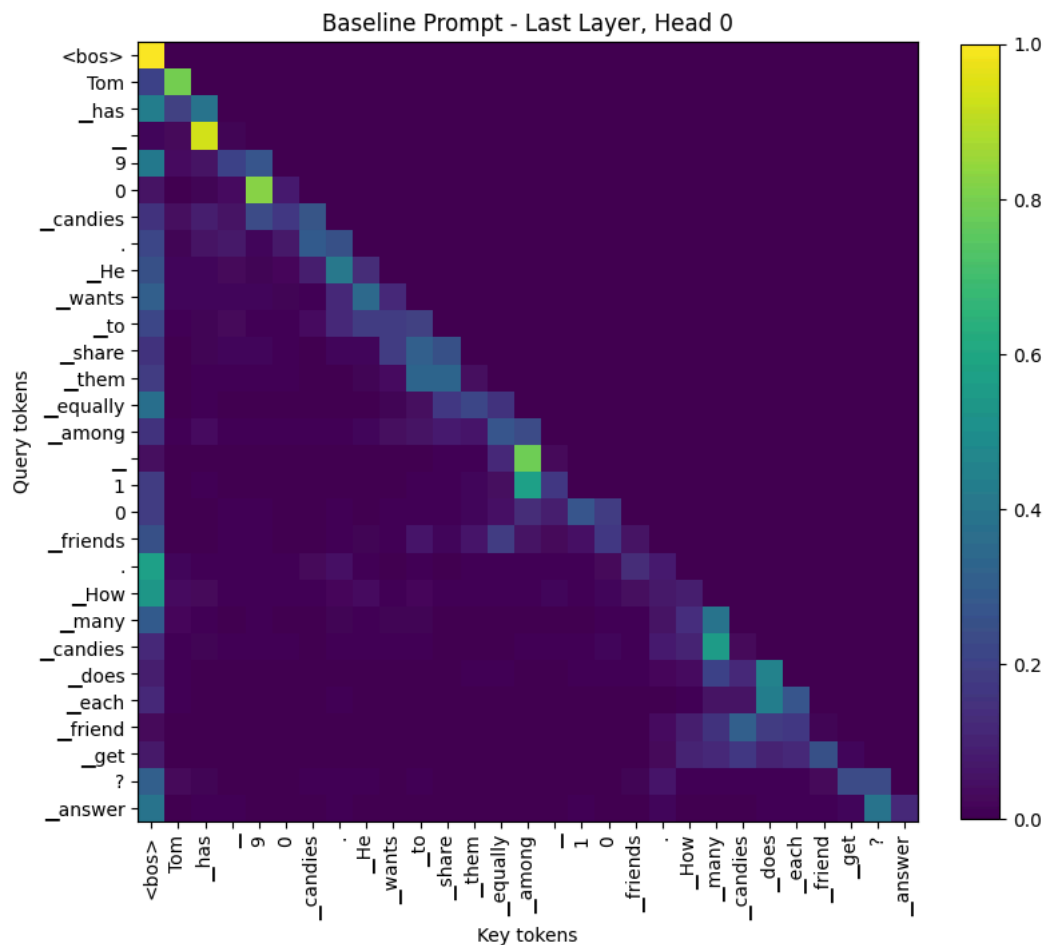
```


Analyzing attention for question:

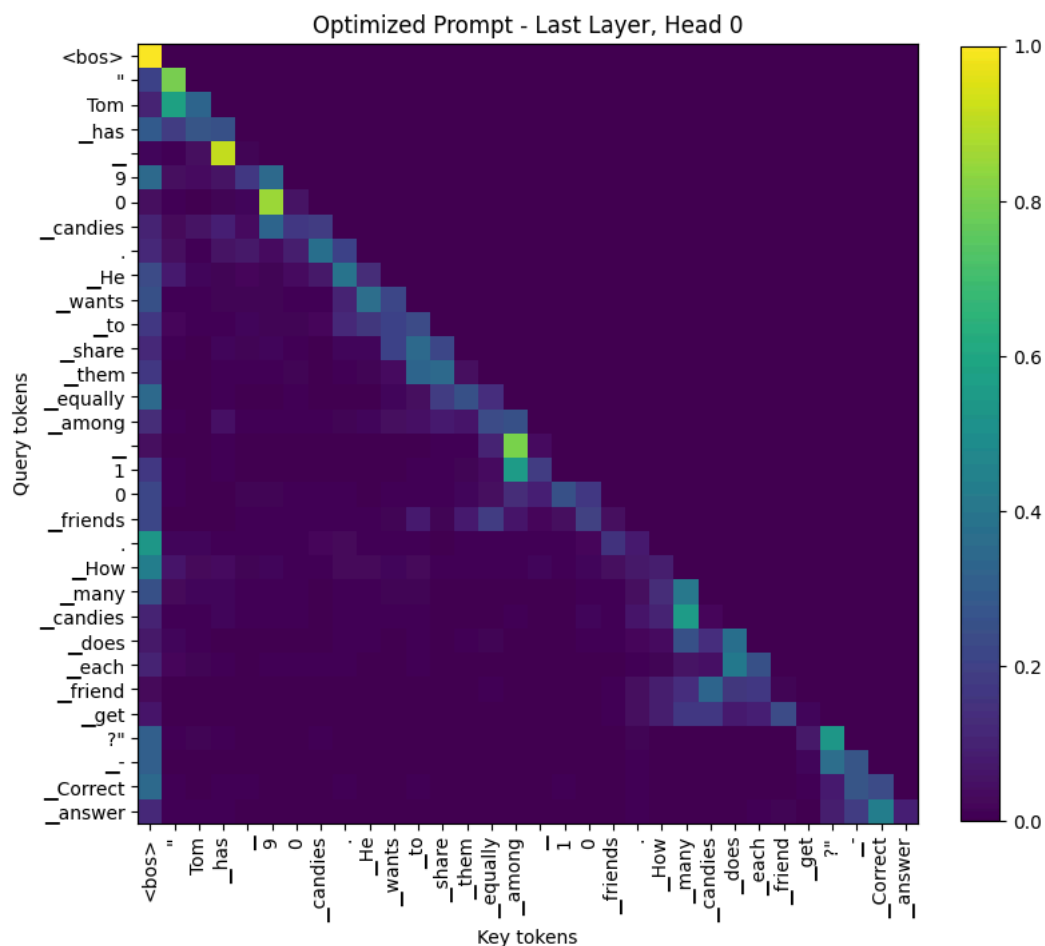
Tom has 90 candies. He wants to share them equally among 10 friends. How many candies does each friend get?

Answer: 9

Baseline prompt: '{question} answer'



Optimized prompt: '"{question}" - Correct answer'



```

1 def compute_attention_concentration(
2     attention: torch.Tensor,
3     tokens: List[str],
4     number_pattern: str = r'\d+'
5 ) -> Dict:
6     """
7     Compute how much attention is concentrated on number tokens.
8
9     Args:
10         attention: Attention tensor (layers, heads, seq, seq)
11         tokens: List of token strings
12         number_pattern: Regex pattern for number tokens
13
14     Returns:
15         Dictionary with attention concentration metrics
16     """
17     ## YOUR_CODE_STARTS_HERE
18     num_idx = [i for i, tok in enumerate(tokens) if re.search(number_pattern, tok)]
19     if not num_idx:
20         return{
21             "num_no_tokens": 0,
22             "avg_attention_per_number": 0.0
23         }
24
25     attn_avg = attention.mean(dim = (0,1))
26     num_attn = attn_avg[:, num_idx].mean().item()
27     total_attn = attn_avg.mean().item()
28     if total_attn > 0:
29         norm_attn = num_attn / total_attn
30     else:
31         norm_attn = 0.0
32
33     return{
34         "num_no_tokens": len(num_idx),
35         "avg_attention_per_number": norm_attn
36     }
37
38     ## YOUR_CODE_ENDS_HERE
39

```

```

1 # Compare attention concentration across multiple examples
2 print("Attention Concentration Analysis")
3 print("="*70)
4 print("Measuring how much attention is directed toward number tokens...\n")
5
6 baseline_number_attention = []
7 optimized_number_attention = []
8
9 for i, example in enumerate(test_data[:10]):
10     # Baseline
11     b_attn, b_tokens = get_encoder_attention(model, tokenizer, best_baseline_prompt, example['question'])
12     b_conc = compute_attention_concentration(b_attn, b_tokens)
13     baseline_number_attention.append(b_conc['avg_attention_per_number'])
14     # Optimized
15     o_attn, o_tokens = get_encoder_attention(model, tokenizer, optimized_prompt, example['question'])
16     o_conc = compute_attention_concentration(o_attn, o_tokens)
17     optimized_number_attention.append(o_conc['avg_attention_per_number'])
18 print(f"Average Attention per Number Token:")
19 print(f"  Baseline:  {np.mean(baseline_number_attention):.4f}")
20 print(f"  Optimized: {np.mean(optimized_number_attention):.4f}")
21 print()
22
23 # Visualize
24 fig, ax = plt.subplots(figsize=(10, 5))
25 x = np.arange(10)
26 width = 0.35
27 ax.bar(x - width/2, baseline_number_attention, width, label='Baseline', alpha=0.8)
28 ax.bar(x + width/2, optimized_number_attention, width, label='Optimized', alpha=0.8)
29 ax.set_xlabel('Example Index')
30 ax.set_ylabel('Avg Attention per Number Token (normalized)')
31 ax.set_title('Attention Concentration on Numbers: Baseline vs Optimized')
32 ax.legend()
33 ax.set_xticks(x)
34 plt.tight_layout()
35 plt.show()
36

```

Attention Concentration Analysis

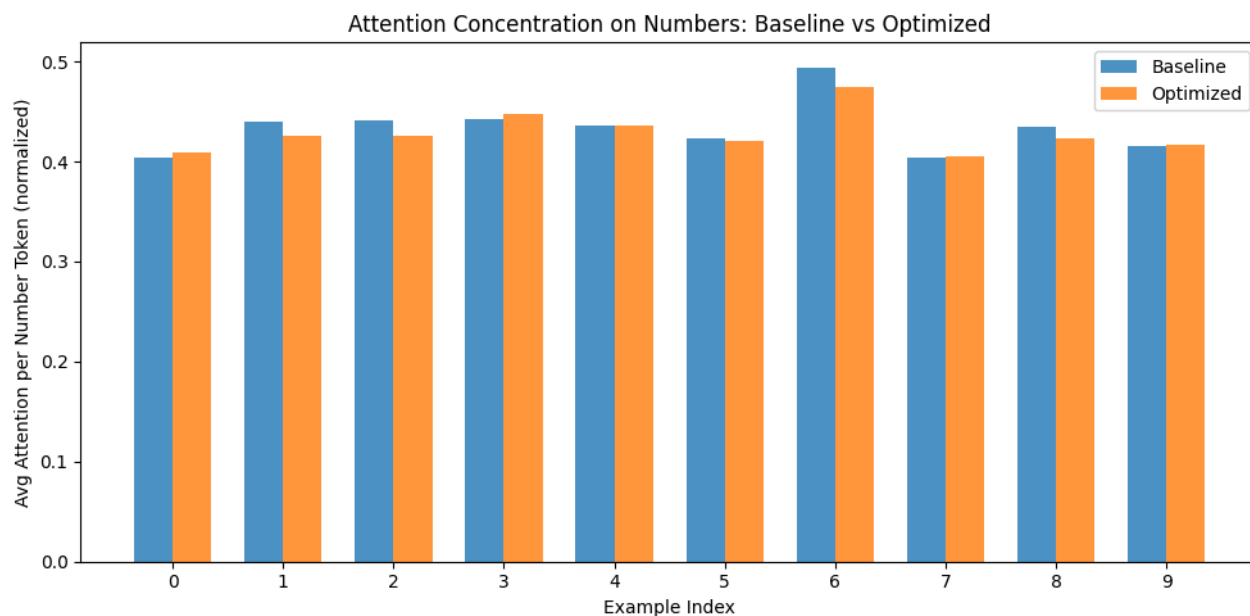
=====

Measuring how much attention is directed toward number tokens...

Average Attention per Number Token:

Baseline: 0.4336

Optimized: 0.4285



Summary and Analysis

Write your analysis of why the optimized prompt performed better based on the interpretability findings.

Your Analysis (Fill this in)

1. Prompt Optimization Results:

- Initial baseline prompt: "{question} answer", accuracy = 75%
- Final optimized prompt: "'{question}' - Correct answer", accuracy = 95%
- Accuracy improvement: 20%

2. Confidence Analysis Findings:

- How did model confidence change with the optimized prompt?

Both the baseline and optimized prompts produced the same first-token attention score. As for the entropy, it reached a lower value compared to the baseline. These results show that the model shows more generation certainty optimized.

- What does the entropy comparison tell us?

Baseline: 1.4999

Optimized: 0.4897

Entropy is significantly lowered in the optimized version, which shows that uncertainty was reduced and more confidence was put on more likely tokens.

3. Attention Analysis Findings:

- Did the optimized prompt change what the model attends to?

In the second attention map the attention line is more strongly aligned and fit, differently from the baseline one, where towards the end of the sequence where the answer is to be generated, the attention is more scattered.

- Was there more attention on important tokens (numbers)?

Baseline: Average attention per number token = 0.4336

Optimized: 0.4285

Increased average attention per number shows that the model is reasoning better about the specific arithmetic task. However, in optimized case, attention to number tokens is lower, which indicates that the model is putting more attention to the construct and task needed to be solved in the sentence than only the mathematical digits.

4. Why Did Optimization Work?

- Based on your analysis, explain why the optimized prompt performs better.

Optimization worked because it forced the model to focus on the task by producing a correct expected answer as mentioned in the prompt. The objective and structure of the answer was more clarified, leading to lower output entropy, overall improving accuracy.

5. Limitations:

- What are the limitations of this approach?

Improvements are small, even though the model performs well and more confidently under the specified task. However, this does not guarantee correction for other tasks.

- When might automatic prompt optimization fail?

It fails for more complex tasks, where more reasoning steps are required or in cases where a well-defined answer structure is not sufficient for to achieve correctness.

```
1 # Final summary statistics
2 print("="*70)
3 print("FINAL SUMMARY")
4 print("="*70)
5 print(f"\nBaseline Prompt: '{best_baseline_prompt}'")
6 print(f"Optimized Prompt: '{optimized_prompt}'")
7 print(f"\nTest Set Performance:")
8 print(f"  Baseline Accuracy: {baseline_test_acc:.2%}")
9 print(f"  Optimized Accuracy: {optimized_test_acc:.2%}")
10 print(f"  Improvement: {(optimized_test_acc - baseline_test_acc)*100:+.1f}%")
11 print(f"\nInterpretability Metrics (averaged over test examples):")
12 print(f"  Confidence - Baseline: {np.mean(baseline_confidences):.4f}")
13 print(f"  Confidence - Optimized: {np.mean(optimized_confidences):.4f}")
14 print(f"  Avg Attention per Number Token - Baseline: {np.mean(baseline_number_attention):.4f}")
15 print(f"  Avg Attention per Number Token - Optimized: {np.mean(optimized_number_attention):.4f}")
```

```
=====
FINAL SUMMARY
=====
```

```
Baseline Prompt: '{question} answer'
Optimized Prompt: '"{question}" - Correct answer'
```

```
Test Set Performance:
  Baseline Accuracy: 75.00%
  Optimized Accuracy: 95.00%
  Improvement: +20.0%
```

```
Interpretability Metrics (averaged over test examples):
  Confidence - Baseline: 0.0000
  Confidence - Optimized: 0.0000
  Avg Attention per Number Token - Baseline: 0.4336
  Avg Attention per Number Token - Optimized: 0.4285
```

✓ BONUS (2 points): Solve any task with an LLM

Goal.

Pick **one** of the homework tasks (Task 1, Task 2, or Task 3) and solve it using an **LLM**. Provide the **LLM** with the **task description** and any **starter/prerequisite code** it depends on, ask it to generate a complete **code solution** first, then run that **generated code** here in the code notebook yourself. Finally, document what you did and **compare** the LLM's result to your own pipeline.

What to deliver below.

- LLM used** (name + version, e.g., "Llama-3-8B-Instruct", "GPT-x", "Claude-x", "Mistral-x", etc.).
- Prompt(s)** you used.
- LLM output** — copy and paste the generated code.
- Comparison** to your solution: what matches or differs (quantitative or qualitative).
- Reflection**: what the LLM was **good at** vs **bad at**, what it got **right** vs **wrong**.

No code required. You do **not** need to run, share, or submit any code used for the LLM generation. Provide only the deliverables listed above. You may use any LLMs through any interface (API, web UI, local inference).

1. **LLM used:** GPT-5

2. **Prompt(s):**

- Dataset helper code
- Task 1 overall description

- Description of tasks 1.1, 1.2, 1.3

3. LLM output:

```
1 import torch
2 import re
3 from typing import Dict, List, Tuple, Optional
```

```
1 def generate_response(
2     model,
3     tokenizer,
4     prompt_template: str,
5     question: str,
6     device: torch.device,
7     max_new_tokens: int = 32,
8     return_logits: bool = False,
9     return_attentions: bool = False,
10 ) -> Dict:
11     """
12     Generate a model response given a prompt template and question.
13     """
14
15     # 1. Format prompt
16     prompt = prompt_template.format(question=question)
17
18     # 2. Tokenize
19     inputs = tokenizer(
20         prompt,
21         return_tensors="pt",
22         add_special_tokens=True
23     )
24     inputs = {k: v.to(device) for k, v in inputs.items()}
25
26     input_length = inputs["input_ids"].shape[1]
27
28     # 3. Handle pad token
29     pad_token_id = tokenizer.pad_token_id
30     if pad_token_id is None:
31         pad_token_id = tokenizer.eos_token_id
32
33     # 4. Generate (greedy decoding)
34     with torch.no_grad():
35         outputs = model.generate(
36             **inputs,
37             max_new_tokens=max_new_tokens,
38             do_sample=False,          # greedy
39             pad_token_id=pad_token_id,
40             output_logits=return_logits,
41             output_attentions=return_attentions,
42             return_dict_in_generate=True,
43         )
44
45     # 5. Extract only generated tokens
46     generated_tokens = outputs.sequences[:, input_length:]
47
48     # 6. Decode
49     response_text = tokenizer.decode(
50         generated_tokens[0],
51         skip_special_tokens=True
52     ).strip()
53
54     result = {
55         "response": response_text
56     }
57
58     if return_logits:
59         result["logits"] = outputs.logits
60
61     if return_attentions:
62         result["attentions"] = outputs.attentions
63
64     return result
```

```
1 def extract_number(text: str) -> Optional[str]:
2     """
3     Extract the first integer number found in text.
4     Returns the number as a string or None.
5     """
6     if text is None:
7         return None
```

```

8
9     match = re.search(r"-?\d+", text)
10    if match:
11        return match.group(0)
12
13    return None

```

```

1 def evaluate_prompt(
2     model,
3     tokenizer,
4     prompt_template: str,
5     dataset: List[Dict],
6     device: torch.device,
7 ) -> Tuple[float, List[Dict]]:
8     """
9     Evaluate a prompt on a dataset.
10    Returns (accuracy, failed_cases).
11    """
12
13    correct = 0
14    failed_cases = []
15
16    for example in dataset:
17        question = example["question"]
18        target = example["answer"]
19
20        output = generate_response(
21            model=model,
22            tokenizer=tokenizer,
23            prompt_template=prompt_template,
24            question=question,
25            device=device,
26        )
27
28        raw_response = output["response"]
29        prediction = extract_number(raw_response)
30
31        if prediction == target:
32            correct += 1
33        else:
34            failed_cases.append({
35                "question": question,
36                "prediction": prediction,
37                "target": target,
38                "raw_response": raw_response,
39            })
40
41    accuracy = correct / len(dataset)
42
43    return accuracy, failed_cases

```

```

1 baseline_prompts = [
2     "Answer the following question:\n{question}\nAnswer:",
3     "{question}\n",
4     "Solve this math problem:\n{question}\nFinal answer:",
5     "Question: {question}\nAnswer with a number only:",
6     "Compute the answer.\n{question}\nResult:"
7 ]

```

```

1 baseline_results = {}
2
3 for prompt in baseline_prompts:
4     accuracy, failures = evaluate_prompt(
5         model=model,
6         tokenizer=tokenizer,
7         prompt_template=prompt,
8         dataset=val_data,
9         device=device,
10    )
11
12    baseline_results[prompt] = {
13        "accuracy": accuracy,
14        "failures": failures,
15    }
16
17    print("=" * 60)
18    print(f"Prompt: \n{prompt}")
19    print(f"Accuracy: {accuracy:.3f}")

```

```

=====
Prompt:
Answer the following question:
{question}
Answer:
Accuracy: 0.675
=====
Prompt:
{question}

Accuracy: 0.075
=====
Prompt:
Solve this math problem:
{question}
Final answer:
Accuracy: 0.825
=====
Prompt:
Question: {question}
Answer with a number only:
Accuracy: 0.825
=====
Prompt:
Compute the answer.
{question}
Result:

```

```

1 best_prompt = max(
2     baseline_results.items(),
3     key=lambda x: x[1]["accuracy"]
4 )
5
6 best_prompt_template = best_prompt[0]
7 best_prompt_accuracy = best_prompt[1]["accuracy"]
8
9 print("\nBest baseline prompt:")
10 print(best_prompt_template)
11 print(f"Accuracy: {best_prompt_accuracy:.3f}")
12

```

```

Best baseline prompt:
Solve this math problem:
{question}
Final answer:
Accuracy: 0.825

```

4. Comparison:

As for the code generation, due to the specific instructions provided in the descriptions, the code follows the same logic as the tasks required. It also generated the baseline prompts and `extract_number` function on its own, without specifying them from the homework.

It can be seen that the prompts generated by the model are more optimal and involve more instructions compared to the minimal ones in task. For that reason, it mainly achieves above 50% accuracies (without optimization), with the two best ones:

- Solve this math problem: {question} Final answer:
- Question: {question} Answer with a number only:

reaching 82.5% accuracy.

5. Reflection:

In general, the model generated the solution code for the task very well. It followed the steps specified on the prompt, produces good prompts that achieve high accuracy and is able to do the evaluation correctly.

One difference is noticed on the `extract_number` function which, generated by the LLM followed a simpler approach than the one provided in the task, which is more constrained to finding numbers related to the answer.